🏛 GLA UNIVERSITY, MATHURA — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

HPC Masterclass | Day 3 Post-Assessment Question Paper

Course: B.Tech CSE 7th Sem — High Performance Computing Masterclass
Module: Day 3 Mastery — Advanced Deep Learning Systems, oneDNN Operator Fusion, Slurm & Distributed Horovod
Duration: 60 Minutes | **Total Questions**: 50 MCQs | **Maximum Marks**: 50

Student Name: _____ **Roll Number**: _____ **Section**: _____

📝 Instructions for Candidates

1. All questions are compulsory. Each question carries exactly 1 mark.
2. Select the single best answer for each question.
3. No negative marking.

Q01. In Intel oneDNN execution pipelines, what is the architectural distinction between a 'primitive descriptor' and an executable 'primitive'?

- ▢ **(a)** A primitive descriptor stores weights on NVMe disk, while a primitive stores weights in RAM.
- ▢ **(b)** A primitive descriptor defines the mathematical operation, shapes, data types, and engine attributes to query optimal algorithms; a primitive is the instantiated executable object created from the descriptor.
- ▢ **(c)** A primitive descriptor only exists in Python, while a primitive only exists in Fortran.
- ▢ **(d)** A primitive descriptor compiles machine code, while a primitive decompiles it.

Q02. When oneDNN fuses a Conv2D operation with an elementwise ReLU activation and a residual Addition (post-op sum), what specific memory traffic is eliminated?

- ▢ **(a)** The read traffic for model weights from L1 cache.
- ▢ **(b)** Two intermediate DRAM write operations (Conv2D output, Sum output) and two intermediate DRAM read operations (reading back for Addition and ReLU).
- ▢ **(c)** The network transmission of batch labels over InfiniBand.
- ▢ **(d)** The PCIe transfer of dataset files from host to disk.

Q03. What information does setting the environment variable 'ONEDNN_VERBOSE=1' output to the console during neural network execution?

- ▢ **(a)** The entire Python abstract syntax tree (AST).
- ▢ **(b)** A detailed execution log for every oneDNN primitive call, including engine type, primitive kind, implementation name (e.g., jit:avx512_core), memory formats, input/output tensor shapes, and kernel execution time in milliseconds.
- ▢ **(c)** The CPU temperature and fan RPM speeds.

- ▢ **(d)** The operating system user password hashes.

Q04. Why does oneDNN introduce 'blocked memory layouts' such as 'nChw16c' instead of standard contiguous NCHW or NHWC on AVX-512 enabled Intel Xeon processors?

- ▢ **(a)** To encrypt tensor activations to meet security compliance.
- ▢ **(b)** To chunk channels into blocks of 16 contiguous elements matching 512-bit vector register capacity (16 floats x 32 bits = 512 bits), enabling full vector FMA utilization without strided memory gathering.
- ▢ **(c)** To convert 4-dimensional tensors into 2-dimensional CSV spreadsheets.
- ▢ **(d)** To ensure compatibility with legacy 8-bit microprocessors.

Q05. What is 'weight pre-packing' (weight reordering) in oneDNN inference pipelines, and why is it beneficial for deployment?

- ▢ **(a)** Compressing weight files with gzip before uploading to the server.
- ▢ **(b)** Transforming static model weights offline from standard user layout (e.g., OHWI) into the hardware-optimized blocked format (e.g., OIhw16i16o) once at load time, avoiding per-inference layout reordering overhead.
- ▢ **(c)** Converting all model weights into random numbers to test stability.
- ▢ **(d)** Moving weights from RAM into external USB storage.

Q06. In Intel AMX (Advanced Matrix Extensions), how do the 8 hardware 2D tile registers (TMM0 to TMM7) organize data for the TMUL (Tile Matrix Multiply) unit?

- ▢ **(a)** Each tile holds a 1D scalar variable of 64 bits.
- ▢ **(b)** Each tile holds a 2D matrix chunk of up to 16 rows and 64 bytes per row (up to 1024 bytes per tile), allowing the TMUL systolic unit to perform dense matrix multiply-accumulate operations directly from registers.
- ▢ **(c)** Each tile stores an entire operating system process table.
- ▢ **(d)** Tiles can only store 32-bit integer counters.

Q07. What does oneDNN's 'scratchpad memory' management allow an HPC developer or framework integrator to optimize?

- ▢ **(a)** It allows allocating temporary working memory buffers from a custom user memory pool rather than having the library call dynamic 'malloc/free' inside high-frequency execution loops.
- ▢ **(b)** It provides a text editor inside the Linux terminal.
- ▢ **(c)** It formats the swap partition on the boot drive.
- ▢ **(d)** It writes log messages to systemd journal.

Q08. What low-level hardware instruction set is utilized by oneDNN on 2nd and 3rd Gen Intel Xeon Scalable processors (Cascade Lake / Ice Lake) to achieve $4\times$ throughput on INT8 quantized neural networks?

- ▢ **(a)** Intel MMX
- ▢ **(b)** AVX-512 VNNI (Vector Neural Network Instructions) using `VPDPBUSD`
- ▢ **(c)** ARM NEON
- ▢ **(d)** RISC-V Vector Extension

Q09. When configuring thread affinity for oneDNN on a dual-socket Intel Xeon system with 64 physical cores, which OpenMP configuration prevents thread migration and false sharing?

- ▢ **(a)** OMP_NUM_THREADS=1000; OMP_PLACES=threads
- ▢ **(b)** OMP_NUM_THREADS=64; OMP_PLACES=cores; OMP_PROC_BIND=close
- ▢ **(c)** OMP_DYNAMIC=TRUE; OMP_NESTED=TRUE
- ▢ **(d)** OMP_WAIT_POLICY=PASSIVE; OMP_NUM_THREADS=1

Q10. Why does oneDNN prefer executing JIT-compiled kernels (Just-In-Time) over pre-compiled static library functions for deep learning primitives?

- ▢ **(a)** Static libraries cannot run on Linux servers.
- ▢ **(b)** JIT compilation inspects the exact tensor dimensions, strides, padding, and specific CPU microarchitecture at runtime, emitting assembly instructions with hardcoded loop bounds and fully unrolled vector registers.
- ▢ **(c)** JIT compilation eliminates the need for physical CPU caches.
- ▢ **(d)** JIT kernels do not require any memory allocation.

Q11. In the Intel Xe-HPC architecture (Intel Data Center GPU Max), what is the function of the Xe Matrix Extension (XMX) systolic engines within each Xe Core?

- ▢ **(a)** Handling PCIe root complex interrupts.
- ▢ **(b)** Executing dense matrix multiply-accumulate operations (DPAS - Dot Product Accumulate Systolic) at high arithmetic throughput for FP16, BF16, and INT8 tensors.
- ▢ **(c)** Decompressing video streams for web browsers.
- ▢ **(d)** Managing hard drive RAID controllers.

Q12. What is 'Xe Link' in multi-GPU Intel Data Center GPU Max server deployments?

- ▢ **(a)** A web browser extension for monitoring GPU clock speeds.
- ▢ **(b)** A high-speed, direct inter-GPU interconnect fabric that enables coherent memory access and high-bandwidth peer-to-peer communication between GPU tiles without traversing the host PCIe bus.
- ▢ **(c)** A serial RS-232 cable for debugging kernel panics.
- ▢ **(d)** A software emulator for running CUDA code on AMD GPUs.

Q13. How does Intel Extension for TensorFlow (ITEX) interface with Intel GPUs under the hood?

- ▢ **(a)** By translating TensorFlow graphs into Java bytecode.
- ▢ **(b)** Via standard oneAPI Level Zero driver interfaces and SYCL runtime kernels registered through the TensorFlow PluggableDevice C API.
- ▢ **(c)** By running an embedded virtual machine inside the GPU firmware.
- ▢ **(d)** Via legacy OpenGL 1.1 graphics pipelines.

Q14. What are the two distinct types of compute cores inside the Habana Gaudi processor architecture?

- ▢ **(a)** ALU and FPU
- ▢ **(b)** Matrix Multiplication Engine (MME) and Tensor Processor Cores (TPC)
- ▢ **(c)** CPU core and GPU shader core

- ▢ **(d)** Cortex-A53 and Cortex-A72

Q15. How does Habana Gaudi's integration of 24 on-die 100GbE RoCE ports per processor revolutionize multi-node distributed training clusters?

- ▢ **(a)** It allows connecting 24 computer monitors directly to each processor.
- ▢ **(b)** It enables direct inter-processor scale-out communication across standard Ethernet switches without needing expensive proprietary NICs (such as PCIe InfiniBand adapters).
- ▢ **(c)** It forces all network packets to travel over Wi-Fi.
- ▢ **(d)** It limits cluster size to a maximum of 2 nodes.

Q16. What is the function of the Habana Communication Library (HCL) within the SynapseAI software stack?

- ▢ **(a)** Displaying chat messages between developers.
- ▢ **(b)** Providing high-performance collective communication primitives (AllReduce, AllGather, Broadcast) optimized to exploit Gaudi's on-chip 24x 100GbE RoCE fabric directly.
- ▢ **(c)** Translating Spanish documentation into English.
- ▢ **(d)** Managing user email notifications for failed Slurm jobs.

Q17. Which Python library and module must be loaded in PyTorch or TensorFlow scripts to enable execution on Habana Gaudi accelerators?

- ▢ **(a)** import torch_cuda
- ▢ **(b)** import habana_frameworks.torch.core as htcore
- ▢ **(c)** import gaudi_hardware_driver
- ▢ **(d)** import tensorflow_directx

Q18. What is 'Lazy Mode' execution in the Habana SynapseAI PyTorch/TensorFlow bridge, and why is it preferred over Eager Mode on Gaudi processors?

- ▢ **(a)** Lazy Mode slows down the processor clock to save electricity during training.
- ▢ **(b)** Lazy Mode accumulates operations into a computational graph before triggering execution, allowing the SynapseAI graph compiler to perform operator fusion, memory planning, and MME/TPC pipeline parallelization.
- ▢ **(c)** Lazy Mode executes only odd-numbered layers of the neural network.
- ▢ **(d)** Lazy Mode stores weights in temporary text files.

Q19. What command-line utility provides real-time hardware telemetry (utilization, HBM memory usage, power, temperature) for Habana Gaudi processors on Linux?

- ▢ **(a)** nvidia-smi
- ▢ **(b)** hl-smi
- ▢ **(c)** intel-gpu-tools
- ▢ **(d)** rocm-smi

Q20. In an 8-card Habana Gaudi server, how are the 24 RoCE 100GbE ports per OAM card typically partitioned between intra-node and inter-node fabrics?

- ▢ **(a)** All 24 ports are connected to an external home router.

- ▢ **(b)** 21 ports per card are wired in an all-to-all non-blocking mesh across the 8 internal cards; 3 ports per card (24 ports total per server = 2.4 Tbps) route to external cluster switches.
- ▢ **(c)** 1 port is used for data; 23 ports are left disconnected for cooling airflow.
- ▢ **(d)** Ports are dynamically reassigned as audio channels.

Q21. In an enterprise Slurm script for distributed deep learning across 4 nodes with 4 GPUs each, which combination of directives correctly configures the job?

- ▢ **(a)** #SBATCH --nodes=4 #SBATCH --ntasks-per-node=4 #SBATCH --gpus-per-node=4
- ▢ **(b)** #SBATCH --nodes=1 #SBATCH --ntasks=1 #SBATCH --gpus=16
- ▢ **(c)** #SBATCH --partition=all #SBATCH --threads=100
- ▢ **(d)** #SBATCH --nodes=16 #SBATCH --exclusive=no #SBATCH --time=unlimited

Q22. What is the consequence of failing to set '#SBATCH --cpus-per-task' appropriately in a multi-GPU deep learning Slurm batch script?

- ▢ **(a)** The job script will fail to compile Python code.
- ▢ **(b)** Slurm defaults to allocating only 1 CPU core per GPU task, causing intense CPU data loading and preprocessing bottlenecks that starve the GPUs and cause massive idle stalls.
- ▢ **(c)** The GPUs will automatically run at half voltage.
- ▢ **(d)** The InfiniBand network adapter will disable RDMA.

Q23. What information does the Slurm command 'sacct -j --format=JobID,JobName,AllocCPUS,Elapsed,State,MaxRSS,ExitCode' provide to an HPC engineer?

- ▢ **(a)** A forecast of the electricity bill for the server.
- ▢ **(b)** Detailed accounting records for the completed or failed job, including peak resident memory consumption (MaxRSS), total CPU count, runtime duration, exit status, and termination reason.
- ▢ **(c)** The source code of the Python script submitted.
- ▢ **(d)** The list of other users logged in via SSH.

Q24. When a Slurm job fails with exit code 137 (SIGKILL) and state 'OUT_OF_MEMORY', what occurred during execution?

- ▢ **(a)** The GPU driver crashed due to cosmic ray bit flips.
- ▢ **(b)** The Linux kernel Out-Of-Memory (OOM) killer or Slurm's cgroup memory enforcement subsystem terminated the job for exceeding its requested '#SBATCH --mem' allocation.
- ▢ **(c)** The user manually cancelled the job using 'scancel'.
- ▢ **(d)** The Slurm controller ran out of disk space on the head node.

Q25. In a distributed Slurm environment, why is it recommended to launch multi-node MPI/Horovod applications using 'srun' rather than standalone 'mpirun'?

- ▢ **(a)** Because mpirun cannot run C++ programs.
- ▢ **(b)** Because 'srun' directly queries the Slurm job step manager, automatically inheriting node allocations, task counts, CPU affinity masks, and GRES GPU bindings without requiring manual hostfile creation.
- ▢ **(c)** Because srun bypasses Linux user permissions.

- ▢ **(d)** Because mpirun disables high-speed InfiniBand interfaces.

Q26. What does the Slurm directive '#SBATCH --gres=gpu:4' specify to the resource manager?

- ▢ **(a)** That the job requires 4 gigabytes of network buffer.
- ▢ **(b)** That the job requests 4 Generic Resources (GRES) of type GPU on the allocated node.
- ▢ **(c)** That the job must run for exactly 4 minutes.
- ▢ **(d)** That 4 different users will share the job output.

Q27. How does Slurm enforce CPU core pinning and memory locality across NUMA domains for distributed deep learning tasks?

- ▢ **(a)** By using the '--cpu-bind=cores' and '--mem-bind=local' directives combined with Linux cgroups.
- ▢ **(b)** By rebooting the server into single-user mode.
- ▢ **(c)** By re-soldering the CPU sockets onto the motherboard.
- ▢ **(d)** By disabling the L3 cache in BIOS.

Q28. What does a pending job status with 'Reason: Priority' in 'squeue' indicate?

- ▢ **(a)** The job failed with a syntax error.
- ▢ **(b)** The job is waiting in the queue because higher-priority jobs (determined by Multifactor Priority policies like fair-share, job age, and partition weight) are scheduled ahead of it.
- ▢ **(c)** The compute nodes are broken and undergoing repair.
- ▢ **(d)** The user forgot to provide a script filename.

Q29. Why is an HPC cluster administrator's 'Prolog' and 'Epilog' script configuration crucial for multi-user deep learning clusters?

- ▢ **(a)** To print welcome poetry when users log in.
- ▢ **(b)** To sanitize the node before and after each job: verifying GPU health, flushing residual GPU memory, resetting PCIe clocks, and terminating orphaned user processes to prevent state contamination.
- ▢ **(c)** To convert markdown files to PDF automatically.
- ▢ **(d)** To backup user home directories to tape drives.

Q30. In a Slurm heterogeneous job or array job, what does the environment variable '$SLURM_ARRAY_TASK_ID' represent?

- ▢ **(a)** The total number of tasks in the entire cluster.
- ▢ **(b)** The specific unique index of the current sub-task within a Slurm job array, allowing one batch script to process different hyperparameter combinations or data shards.
- ▢ **(c)** The process ID of the Slurm daemon.
- ▢ **(d)** The version of Python installed on the node.

Q31. In the Ring-AllReduce algorithm, how many logical phases are executed, and what are their names?

- ▢ **(a)** 1 phase: Direct Broadcast.
- ▢ **(b)** 2 phases: Scatter-Reduce (where each worker accumulates a slice of the final sum) followed by AllGather (where the accumulated slices are distributed to all workers).

- ▢ **(c)** 3 phases: Map, Shuffle, and Reduce.
- ▢ **(d)** 4 phases: Encrypt, Compress, Send, and Decrypt.

Q32. Why is the communication time in Ring-AllReduce described as 'bandwidth-optimal' and independent of the number of worker nodes N?

- ▢ **(a)** Because data is transmitted through quantum entanglement.
- ▢ **(b)** Because the total data transferred by each worker per step is $2 \times \frac{N-1}{N} \times M$, which asymptotically approaches a constant $2M$ as N becomes large, meaning network bandwidth per node does not degrade as the cluster scales.
- ▢ **(c)** Because all workers share a single physical hard drive.
- ▢ **(d)** Because gradients are never transferred across the network.

Q33. In Horovod, why must dataset sharding be implemented via 'dataset.shard(num_shards=hvd.size(), index=hvd.rank())'?

- ▢ **(a)** To compress image files into zip archives.
- ▢ **(b)** To ensure that each distributed worker processes a mutually disjoint, unique partition of the training dataset, avoiding redundant duplicate gradient computation.
- ▢ **(c)** To convert images into audio files.
- ▢ **(d)** To encrypt the training data across workers.

Q34. In a distributed training script using Horovod with TensorFlow, why is the checkpoint saving code wrapped in 'if hvd.rank() == 0:'?

- ▢ **(a)** Because worker ranks other than 0 do not have access to model weights.
- ▢ **(b)** To prevent all N concurrent workers from simultaneously writing to the exact same checkpoint file path on the shared filesystem, which would cause severe file lock corruption and I/O thrashing.
- ▢ **(c)** Because rank 0 is the only process with a Python interpreter.
- ▢ **(d)** To delete model weights after training finishes.

Q35. What is 'Tensor Fusion' in Horovod, controlled by the environment variable 'HOROVOD_FUSION_THRESHOLD'?

- ▢ **(a)** Combining two physical GPUs into one single GPU die.
- ▢ **(b)** A performance feature that packs multiple small gradient tensors into a single contiguous fusion buffer before calling Ring-AllReduce, amortizing network transfer latency.
- ▢ **(c)** A technique to fuse text and image models together.
- ▢ **(d)** A compiler pass that deletes unused convolutional filters.

Q36. What occurs if a developer forgets to call 'hvd.broadcast_variables(model.variables, root_rank=0)' at the start of Horovod training?

- ▢ **(a)** The script will immediately raise a SyntaxError.
- ▢ **(b)** Each worker starts with different randomly initialized weights; averaging gradients computed from divergent parameter spaces causes the model to fail to converge and produce garbage predictions.
- ▢ **(c)** The computer will reboot automatically.
- ▢ **(d)** The dataset will be deleted from the filesystem.

Q37. How does Horovod's 'Auto-Tuning' feature (`--autotune`) optimize distributed training performance during runtime?

- ▢ **(a)** By automatically rewriting the neural network architecture in PyTorch.
- ▢ **(b)** By dynamically searching for the optimal values of fusion threshold, cycle time, and response cache size on the active cluster fabric during early training epochs.
- ▢ **(c)** By increasing the GPU fan speed to 100%.
- ▢ **(d)** By downloading pre-trained weights from Hugging Face.

Q38. When scaling from 1 worker to 64 workers with Horovod, why is 'Gradual Learning Rate Warmup' recommended alongside the Linear Scaling Rule?

- ▢ **(a)** To allow the GPU silicon to warm up to operating temperature.
- ▢ **(b)** Because multiplying the learning rate by 64 immediately at step 0 causes severe gradient instability and divergence, as initial random weights produce large, volatile gradients.
- ▢ **(c)** To ensure that all worker nodes connect to the internet.
- ▢ **(d)** Because Slurm kills jobs that use high learning rates early on.

Q39. In Horovod, what is the role of the 'Response Cache' feature?

- ▢ **(a)** Storing web page responses in a local Redis cache.
- ▢ **(b)** Caching tensor metadata and collective negotiation decisions between ranks across iterations to reduce coordinator rank CPU overhead.
- ▢ **(c)** Caching JPEG images in uncompressed bitmap format.
- ▢ **(d)** Storing student test scores in an SQLite table.

Q40. Which command correctly launches a Horovod training script across 4 nodes with 4 GPUs each (16 total processes) using the 'horovodrun' wrapper?

- ▢ **(a)** horovodrun -np 16 -H server1:4,server2:4,server3:4,server4:4 python train.py
- ▢ **(b)** python train.py --horovod --nodes=4
- ▢ **(c)** srun -n 1 python train.py
- ▢ **(d)** mpirun -np 1 train.py

Q41. What is the primary difference between 'Weak Scaling' and 'Strong Scaling' when benchmarking distributed deep learning clusters?

- ▢ **(a)** Weak scaling uses low-cost GPUs; strong scaling uses high-end accelerators.
- ▢ **(b)** In Strong Scaling, the total global batch size is kept constant as nodes increase (testing time-to-solution reduction); in Weak Scaling, the batch size per node is kept constant as nodes increase (total problem size grows proportionally with compute).
- ▢ **(c)** Strong scaling applies only to convolutional networks; weak scaling applies only to Transformers.
- ▢ **(d)** Weak scaling does not use network synchronization.

Q42. In distributed deep learning, why does Strong Scaling inevitably hit an upper scaling limit where adding more nodes yields zero or negative speedup?

- ▢ **(a)** Because Linux cannot support more than 16 IP addresses.

- ▢ **(b)** As N increases, the local batch size per worker (B/N) shrinks, causing local arithmetic intensity to drop into memory-bandwidth bound territory while communication latency overhead begins to dominate step time.
- ▢ **(c)** Because neural networks forget their weights when distributed across nodes.
- ▢ **(d)** Because Python limits floating point operations to 1 billion per day.

Q43. How does 'Communication-Computation Overlap' in modern distributed frameworks (e.g., Horovod, PyTorch DDP) boost scaling efficiency?

- ▢ **(a)** By running data loading in the background while the computer is turned off.
- ▢ **(b)** By triggering the AllReduce gradient synchronization of final layers immediately in the background while earlier layers are still actively executing their backward pass computations.
- ▢ **(c)** By skipping the backward pass entirely on even-numbered epochs.
- ▢ **(d)** By converting all 32-bit floats into 1-bit booleans.

Q44. If an HPC cluster monitoring profile indicates that GPU compute utilization is only 18% during distributed training, what is the MOST probable root cause?

- ▢ **(a)** The GPU clock frequency has been locked by the vendor.
- ▢ **(b)** The input data pipeline (CPU data loading, JPEG decompression, augmentation, or slow storage I/O) is starved, causing the GPUs to spend over 80% of their time stalled waiting for incoming batches.
- ▢ **(c)** The model has converged to zero training loss.
- ▢ **(d)** The learning rate is too large.

Q45. What is the purpose of 'Gradient Clipping by Global Norm' (`tf.clip_by_global_norm`) during large-scale distributed training?

- ▢ **(a)** To delete negative numbers from the dataset.
- ▢ **(b)** To prevent exploding gradients from causing numerical overflow (NaN/Inf) and parameter corruption when large effective batch sizes produce large gradient spikes.
- ▢ **(c)** To crop images to a square aspect ratio.
- ▢ **(d)** To limit the maximum number of training epochs to 100.

Q46. In high-performance RoCE v2 networks, why is Priority Flow Control (PFC) configured on enterprise Ethernet switches?

- ▢ **(a)** To allow video streaming to take priority over scientific data.
- ▢ **(b)** To eliminate packet drops by pausing specific traffic classes (IEEE 802.1Qbb) when switch buffer thresholds are approached, fulfilling RDMA's requirement for a lossless link layer.
- ▢ **(c)** To convert IP addresses into domain names automatically.
- ▢ **(d)** To encrypt Ethernet frame payloads.

Q47. What does a sudden loss value of 'NaN' (Not-a-Number) during distributed mixed-precision training typically indicate, and how is it resolved?

- ▢ **(a)** The hard drive is full; resolved by deleting log files.
- ▢ **(b)** Numerical overflow in 16-bit floating point representations (values exceeding 65,504 in FP16); resolved by enabling dynamic loss scaling or transitioning to BF16 (which supports dynamic range up to 10^{38}).

- ▢ **(c)** The network cable has become unplugged; resolved by rebooting switches.
- ▢ **(d)** The model has achieved 100% test accuracy.

Q48. Why is storing deep learning datasets in container formats like TFRecord, HDF5, or WebDataset superior to storing millions of individual raw image files on parallel filesystems like Lustre?

- ▢ **(a)** Because TFRecords make images look sharper.
- ▢ **(b)** Because packaging millions of files into sequential binary chunks converts random small metadata requests into high-throughput contiguous block streaming I/O, saturating Lustre Object Storage Targets (OSTs).
- ▢ **(c)** Because Linux cannot open more than 1000 files in a single directory.
- ▢ **(d)** Because container formats automatically train the model without Python.

Q49. How does 'Pipeline Parallelism' (e.g., GPipe) overcome the memory barrier of models that cannot fit on a single accelerator?

- ▢ **(a)** By storing all model weights on an external network hard drive.
- ▢ **(b)** By partitioning the sequential layers of a deep network across multiple accelerators and dividing the minibatch into smaller microbatches to pipeline execution across stages.
- ▢ **(c)** By converting all model layers into linear equations.
- ▢ **(d)** By running training only on the CPU host memory.

Q50. In enterprise HPC clusters, what is the role of Intel oneAPI Collective Communications Library (oneCCL) in multi-node deep learning?

- ▢ **(a)** It manages user email subscriptions.
- ▢ **(b)** It provides cross-architecture, hardware-accelerated collective communication primitives (Allreduce, Allgather, Reduce-scatter) optimized for Intel Xeon CPUs, Xe GPUs, and high-speed fabrics.
- ▢ **(c)** It is an interactive drawing tool for convolutional neural networks.
- ▢ **(d)** It replaces standard Linux bash commands with PowerShell.